

bit_analysis

July 8, 2025

1 Análisis de eficiencia de bits

Este notebook analiza qué tan parecidos son los bits recibidos `bits_rx` con los bits transmitidos `bits_tx` y presenta gráficas, métricas de rendimiento y las imágenes que no cumplen con el criterio de coincidencia.

```
[1]: import pandas as pd
from pathlib import Path
from IPython.display import display, Image
import matplotlib.pyplot as plt

path = "./results/cable/"
nombre_csv = "resultados_comparacion_cable.csv"
df = pd.read_csv(path + nombre_csv)

# Vista rápida del DataFrame
display(df.head())
print(f"Filas totales: {len(df)}")
```

	num_auto	bw_plate_tx \
0	1	dataset_yolo/autos_bw/auto001.png
1	1	dataset_yolo/autos_bw/auto001.png
2	1	dataset_yolo/autos_bw/auto001.png
3	1	dataset_yolo/autos_bw/auto001.png
4	2	dataset_yolo/autos_bw/auto002.png

	bw_plate_rx	ocr_txt_tx	bits_equal	bit_errors \
0	data-cable/autos_bw/auto001_rep01.png	KOZ.808	True	0
1	data-cable/autos_bw/auto001_rep02.png	KOZ.808	True	0
2	data-cable/autos_bw/auto001_rep03.png	KOZ.808	True	0
3	data-cable/autos_bw/auto001_rep04.png	KOZ.808	True	0
4	data-cable/autos_bw/auto002_rep01.png	MAU.096	True	0

	bits_tx	bits_rx
0	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
1	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
2	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]
3	[0 0 1 ... 0 0 0]	[0 0 1 ... 0 0 0]

```
4 [0 0 0 ... 0 0 0] [0 0 0 ... 0 0 0]
```

Filas totales: 928

1.1 Distribución de errores de bit por imagen

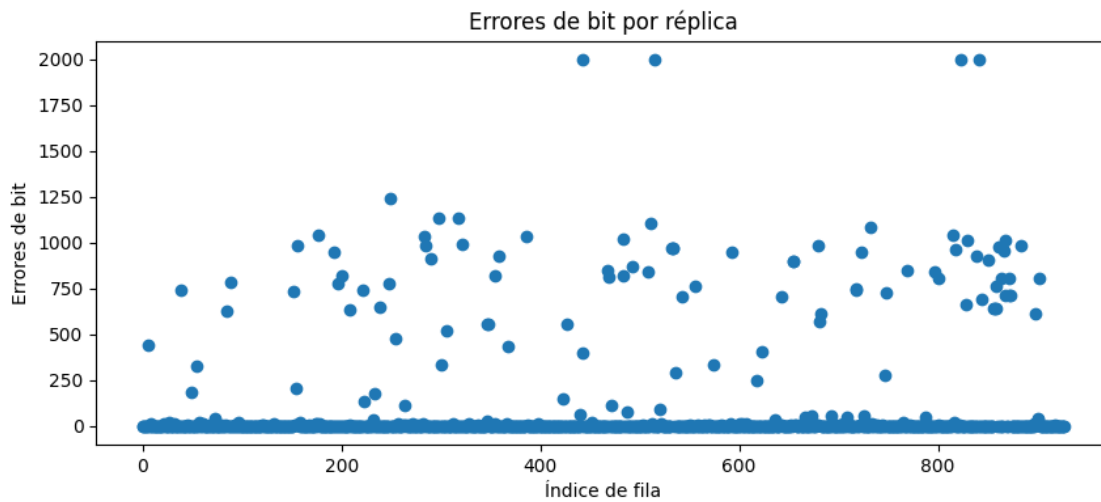
```
[3]: plt.figure()  
df['bit_errors'].plot(kind='hist', bins=30)  
plt.xlabel('Errores de bit por imagen')  
plt.ylabel('Frecuencia')  
plt.title('Histograma de errores de bit')  
plt.show()
```



1.2 Errores de bit por réplica

```
[4]: plt.figure(figsize=(10,4))  
plt.plot(df.index, df['bit_errors'], marker='o', linestyle='none')  
plt.xlabel('Índice de fila')  
plt.ylabel('Errores de bit')  
plt.title('Errores de bit por réplica')
```

```
plt.show()
```



1.3 Imágenes que no cumplen el criterio

```
[24]: # Eliminamos las que tienen 2000 errores, ya que es la imagen inversa
df = df[df['bit_errors'] < 2000]
```

```
[33]: import numpy as np    # por si no estaba importado

# --- Parámetros de tolerancia ---
BITS_TOTAL = 2000
max_errors = 100 # máximo de bits erróneos permitidos
max_ber     = max_errors / BITS_TOTAL # % BER máximo

# --- BER por fila ---
df['bits_total'] = BITS_TOTAL
df['ber'] = df['bit_errors'] / BITS_TOTAL

# --- Criterio de falla ---
fail_df = df[(df['bit_errors'] > max_errors) | (df['ber'] > max_ber)]

print(f'Imágenes que superan el umbral: {len(fail_df)}')
print(f'Total de imágenes: {len(df)}')
print(f'Porcentaje de fallos: {len(fail_df) / len(df) * 100:.2f}%')
display(
    fail_df[['num_auto', 'bw_plate_tx', 'bw_plate_rx',
             'bit_errors', 'ber']]
)
```

Imágenes que superan el umbral: 94

Total de imágenes: 924
 Porcentaje de fallos: 10.17%

	num_auto	bw_plate_tx \		bw_plate_rx	bit_errors	ber
6	2	dataset_yolo/autos_bw/auto002.png				
38	8	dataset_yolo/autos_bw/auto008.png				
49	10	dataset_yolo/autos_bw/auto010.png				
55	10	dataset_yolo/autos_bw/auto010.png				
85	22	dataset_yolo/autos_bw/auto022.png				
..	...					
872	107	dataset_yolo/autos_bw/auto107.png				
873	107	dataset_yolo/autos_bw/auto107.png				
883	41	dataset_yolo/autos_bw/auto041.png				
898	177	dataset_yolo/autos_bw/auto177.png				
902	177	dataset_yolo/autos_bw/auto177.png				

		bw_plate_rx	bit_errors	ber
6	data-cable/autos_bw/auto002_rep03.png		439	0.2195
38	data-cable/autos_bw/auto008_rep04.png		744	0.3720
49	data-cable/autos_bw/auto010_rep03.png		185	0.0925
55	data-cable/autos_bw/auto010_rep09.png		329	0.1645
85	data-cable/autos_bw/auto022_rep01.png		627	0.3135
..	...		...	...
872	data-cable/autos_bw/auto107_rep06.png		713	0.3565
873	data-cable/autos_bw/auto107_rep07.png		713	0.3565
883	data-cable/autos_bw/auto041_rep04.png		987	0.4935
898	data-cable/autos_bw/auto177_rep04.png		609	0.3045
902	data-cable/autos_bw/auto177_rep08.png		803	0.4015

[94 rows x 5 columns]

1.3.1 Visualización de las primeras 5 imágenes fallidas

```
[29]: path = 'results/cable/'

for _, row in fail_df.head(25).iterrows():
    print(f"Auto {row['num_auto']} - errores: {row['bit_errors']}")
    display(Image(filename=row['bw_plate_tx']))
    display(Image(filename=path + row['bw_plate_rx']))
```

Auto 8 - errores: 744



Auto 22 - errores: 627



Auto 19 - errores: 782



Auto 98 - errores: 734



Auto 99 - errores: 986



Auto 38 - errores: 1042



Auto 104 - errores: 949



Auto 74 - errores: 777





Auto 107 - errores: 822



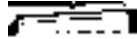
Auto 44 - errores: 637



Auto 111 - errores: 742



Auto 51 - errores: 646



Auto 37 - errores: 776



Auto 37 - errores: 1242



Auto 89 - errores: 1032



Auto 73 - errores: 983



Auto 61 - errores: 913

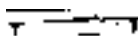


Auto 127 - errores: 1137





Auto 62 - errores: 517



Auto 64 - errores: 1131



Auto 16 - errores: 988



Auto 77 - errores: 557



Auto 77 - errores: 557



Auto 103 - errores: 823



Auto 71 - errores: 925





```
[30]: # --- Umbral de bits erróneos ---
thr_err = 15

# --- Agrupar por auto y comprobar si alguna réplica cumple ---
auto_status = (
    df.groupby('num_auto')['bit_errors']
      .apply(lambda s: (s < thr_err).any())    # True = tiene al menos una buena
)

# Autos sin réplica "buena"
failed_autos = auto_status[~auto_status]      # False significa que falló
num_failed   = failed_autos.size
num_total    = auto_status.size

print(f'Autos sin ninguna réplica con < {thr_err} errores de bit: '
      f'{num_failed} de {num_total}')

# Mostrar la lista (si quieres verla)
display(failed_autos.index.tolist())
```

Autos sin ninguna réplica con < 15 errores de bit: 0 de 182

[]